



ELTE EÖTVÖS LORÁND
UNIVERSITY

Algoritmusok és Adatszerkezetek I.

Pusztai Gábor

Algoritmusok és Alkalmazásaik Tanszék
Informatikai Kar
ELTE - Eötvös Loránd Tudományegyetem, Budapest

2026. február 15.

1 Egyszerű Beszűrő

- "Naiv" Beszűrő rendezés
- Beszűrő rendezés - Példa
- Beszűrő rendezés - Összegzés
- Beszűrő rendezés - Gyakorlat

2 Adatszerkezet - Verem

- Verem (Stack)
- Verem - Típus
- Verem - Műveletek
- Verem - Példa

3 Veremmel megoldható problémák

- Helyes zárójel-szintaxis
- Számláló alapú megoldás
- Zárójelek jelölése

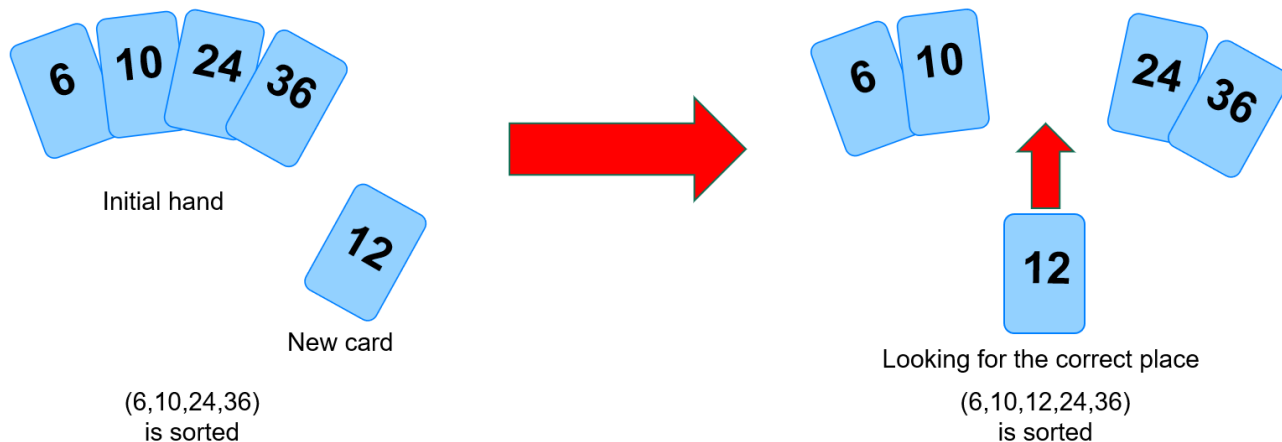
- Nevek fordított sorrendben

4 Lengyel Forma

- Műveletek sorrendje
- Fordított Lengyel Forma
- Átalakítás infixből postfixbe
- Kiértékelése

Egyszerű Beszűrő

Illusztrációs probléma:



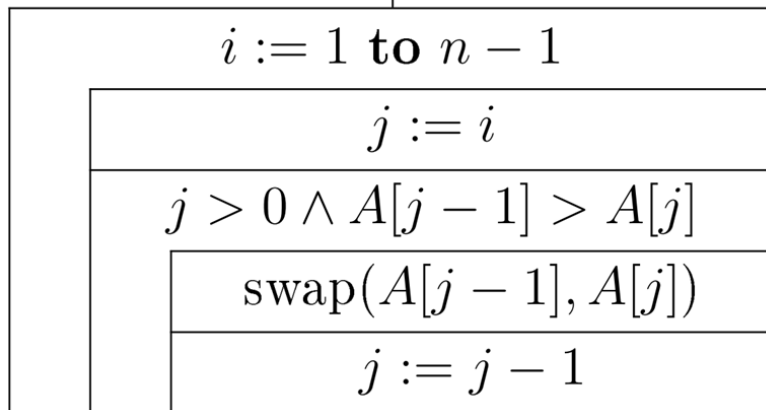
"Naiv" Beszűrő rendezés

	Tömb:	$\langle 5, 4, 2, 8, 3 \rangle$	
Előtte	1. iteráció:	$\langle 5, 4, 2, 8, 3 \rangle$	
Közben	1. iteráció:	$\langle 5 4, 2, 8, 3 \rangle$	
Utána	1. iteráció:	$\langle 5 4, 2, 8, 3 \rangle$	
Előtte	2. iteráció:	$\langle 5 4, 2, 8, 3 \rangle$	
Közben	2. iteráció:	$\langle 5, 4 2, 8, 3 \rangle$	<i>Rendezni kell: cseréljük 5-4-et</i>
Utána	2. iteráció:	$\langle 4, 5 2, 8, 3 \rangle$	
Előtte	3. iteráció:	$\langle 4, 5 2, 8, 3 \rangle$	
Közben	3. iteráció:	$\langle 4, 5, 2 8, 3 \rangle$	<i>Rendezni kell: cseréljük 5-2-t</i>
Közben	3. iteráció:	$\langle 4, 2, 5 8, 3 \rangle$	<i>Cseréljük 4-2-t</i>
Utána	3. iteráció:	$\langle 2, 4, 5 8, 3 \rangle$	
*Utána	4. iteráció:	$\langle 2, 4, 5, 8 3 \rangle$	<i>*részletezés nélkül: 8 már a helyén van</i>
*Utána	5. iteráció:	$\langle 2, 3, 4, 5, 8 \rangle$	<i>*részletezés nélkül: több csere szükséges</i>
Rendezett tömb:		$\langle 2, 3, 4, 5, 8 \rangle$	

"Naiv" Beszúró rendezés

$\langle 5 \mid 4, 2, 8, 3 \rangle$ $i := 1$
Sorted | Unsorted part $A[j] = 4$
 $A[j - 1] = 5$
 $A[j - 1] > A[j] \rightarrow \text{swap}()$

$\text{naiveInsertionSort}(A : \mathcal{T}[n])$



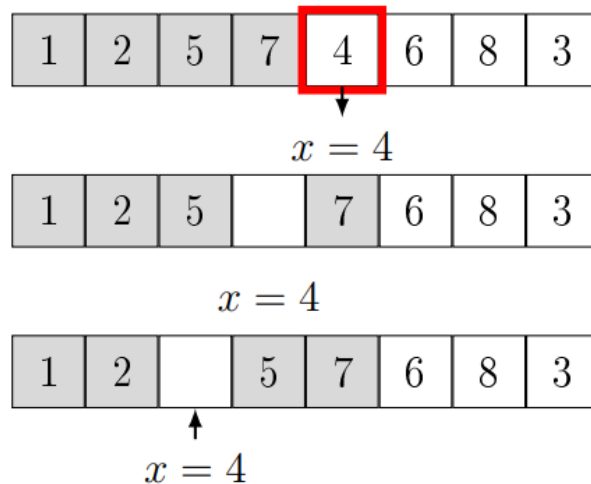
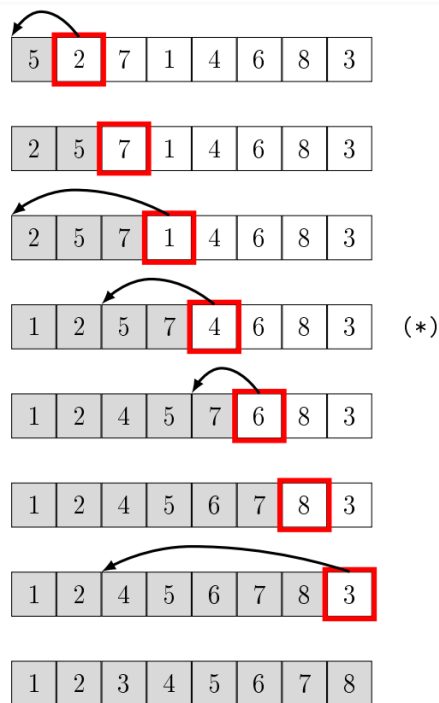
Index: 0 1 2 3 4

$\langle 5, 4, 2, 8, 3 \rangle$
 $\langle 5 \mid 4, 2, 8, 3 \rangle$
 $\langle 4, 5 \mid 2, 8, 3 \rangle$
 $\langle 2, 4, 5 \mid 8, 3 \rangle$
 $\langle 2, 4, 5, 8 \mid 3 \rangle$
 $\langle 2, 4, 5, 3, 8 \mid \rangle$
Detailed swap() $\langle 2, 4, 3, 5, 8 \mid \rangle$
steps $\langle 2, 3, 4, 5, 8 \mid \rangle$
 $\langle 2, 3, 4, 5, 8 \rangle$

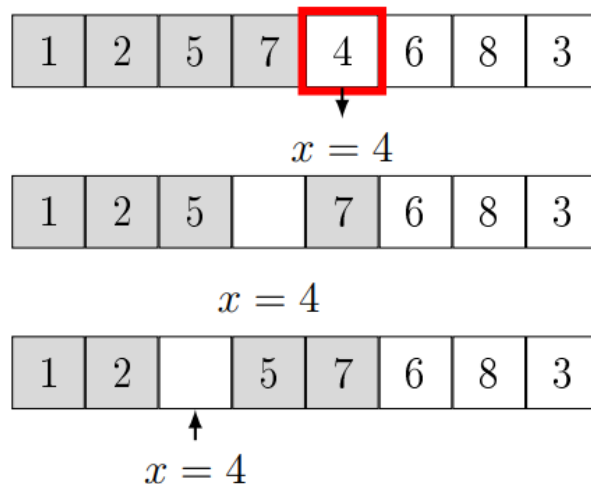
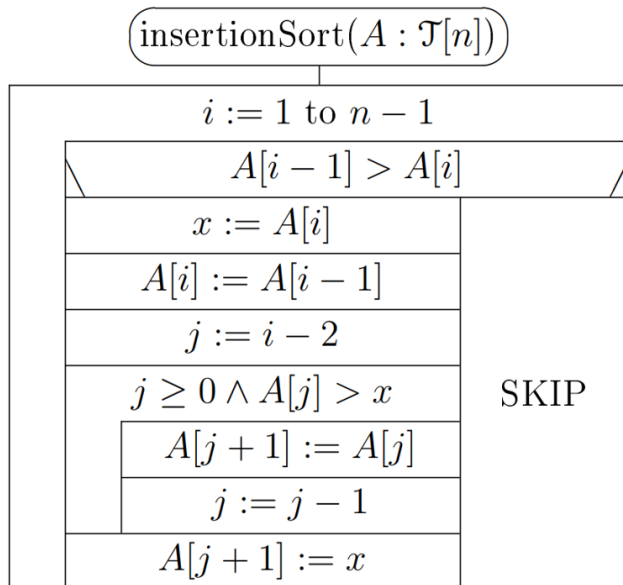
Mi a probléma a naiv Beszűrő rendezéssel?

- A cserék sok számítási kapacitást igényelnek
- Optimalizálás: Csökkentsük a cserék számát
→ *Beszűrő rendezés algoritmus*

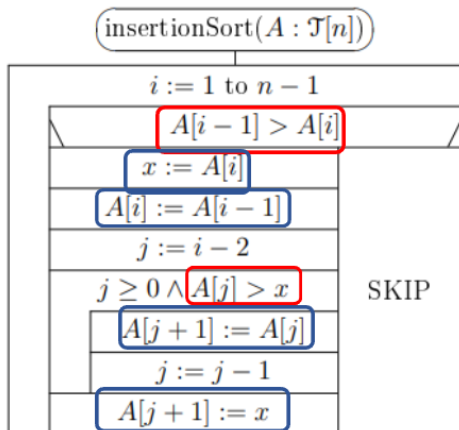
Beszűrő rendezés - Példa



Beszúró rendezés - Példa



Beszúró rendezés - Összegzés



	0	1	2	3	4	5	6	7	comp	move
A	4	3	1	3'	5	2	6	4'		
	0	1	2	3	5	6	7	8		
	>			i						
1. iteration	A	4	3	1	3'	5	2	6	4'	1 3
		>			i					
2. iteration	A	3	4	1	3'	5	2	6	4'	2 4
			£							
			>		i					
3. iteration	A	1	3	4	3'	5	2	6	4'	2 3
				£						
				>						
4. iteration	A	1	3	3'	4	5	2	6	4'	1 0
					£					
				>						
5. iteration	A	1	3	3'	4	5	2	6	4'	5 6
						£				
						>				
6. iteration	A	1	2	3	3'	4	5	6	4'	1 0
							£			
							>			
7. iteration	A	1	2	3	3'	4	5	6	4'	3 4
Overall:	A	1	2	3	3'	4	4'	5	6	15 20

Műveletigénye:

- $mT(n) \in \Theta(n)$
- $AT(n), MT(n) \in \Theta(n^2)$

Tárhely igénye:

- $\Theta(1)$
- Csak néhány ideiglenes változót használ.

Beszűrős rendezés $\rightarrow$ **Stabil**: A rendezési algoritmus megőrzi az azonos értékű elemek eredeti sorrendjét.

Rendezd a következő tömböt *beszúró rendezéssel*!

$\langle 6, 5, 3, 1, 8, 7, 2, 4 \rangle$

Példa:

1. iteráció: $\langle 6 \mid \underline{5}, 3, 1, 8, 7, 2, 4 \rangle$

2. iteráció: $\langle 5, 6 \mid \underline{3}, 1, 8, 7, 2, 4 \rangle$

3. iteráció: $\langle 3, 5, 6 \mid \underline{1}, 8, 7, 2, 4 \rangle$

.....

Rendezd a következő tömböt *beszúró rendezéssel*!

$\langle 6, 5, 3, 1, 8, 7, 2, 4 \rangle$

Megoldás (animáció):

[https://en.wikipedia.org/wiki/Insertion_sort#/media/
File:Insertion-sort-example-300px.gif](https://en.wikipedia.org/wiki/Insertion_sort#/media/File:Insertion-sort-example-300px.gif)

Adatszerkezet - Verem

Verem (Stack)

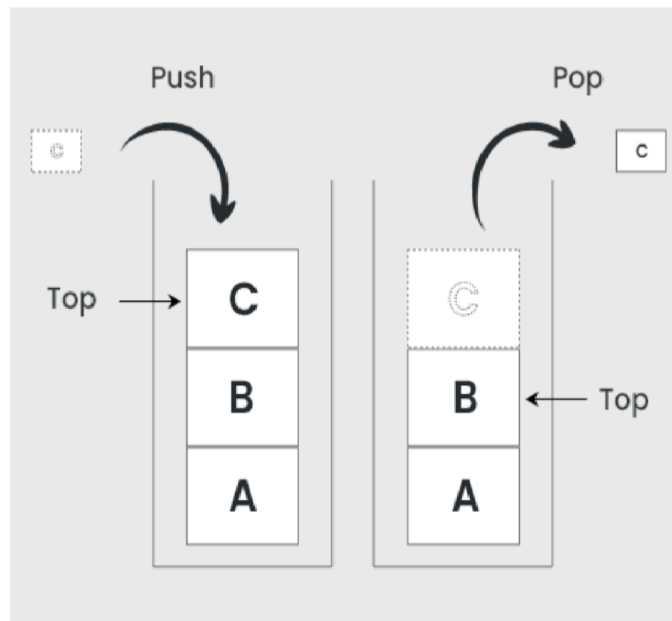
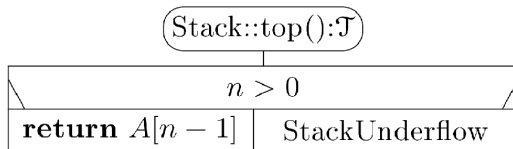
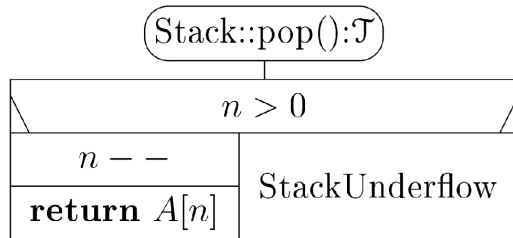
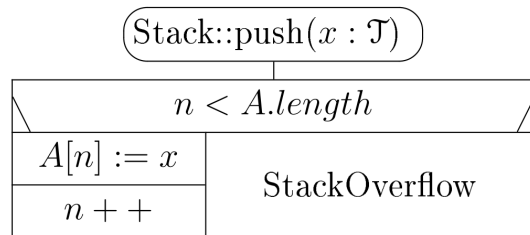
Szemléltető probléma: Most költöztél egy új szobába, és az összes holmid még dobozokban van. Korlátozott a helyed, hogyan tudnál helyet spórolni?

A dobozokat egymás tetejére rakod.

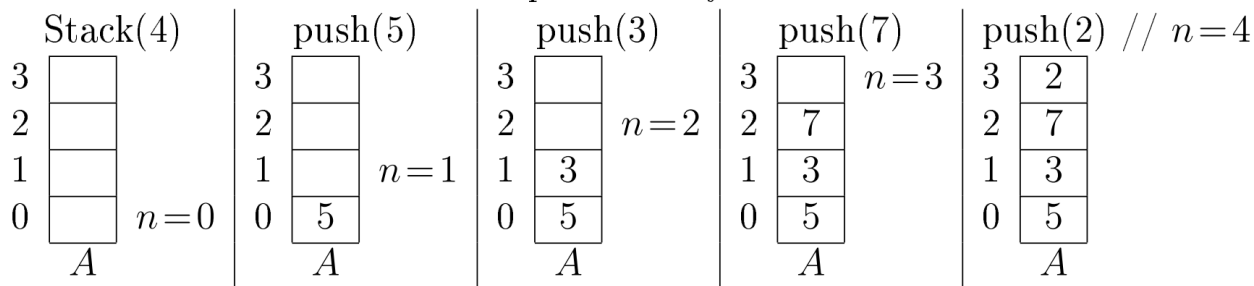
- Behozod az új dobozt a szobába.
- Az előző doboz tetejére helyezed.

Stack
$-A : \mathcal{T}[]$ // $\mathcal{T}$ is some known type ; $A.length$ is the max. size of the stack $-n : \mathbb{N}$ // $n \in 0..A.length$ is the actual size of the stack
$+ \text{Stack}(m : \mathbb{N}) \{ A := \text{new } \mathcal{T}[m] ; n := 0 \}$ // create an empty stack $+ \text{push}(x : \mathcal{T})$ // push x onto the top of the stack $+ \text{pop}() : \mathcal{T}$ // remove and return the top element of the stack $+ \text{top}() : \mathcal{T}$ // return the top element of the stack $+ \text{isFull}() : \mathbb{B} \{ \text{return } n = A.length \}$ $+ \text{isEmpty}() : \mathbb{B} \{ \text{return } n = 0 \}$ $+ \text{setEmpty}() \{ n := 0 \}$ // reinitialize the stack $+ \sim \text{Stack}() \{ \text{delete } A \}$

Verem - Műveletek



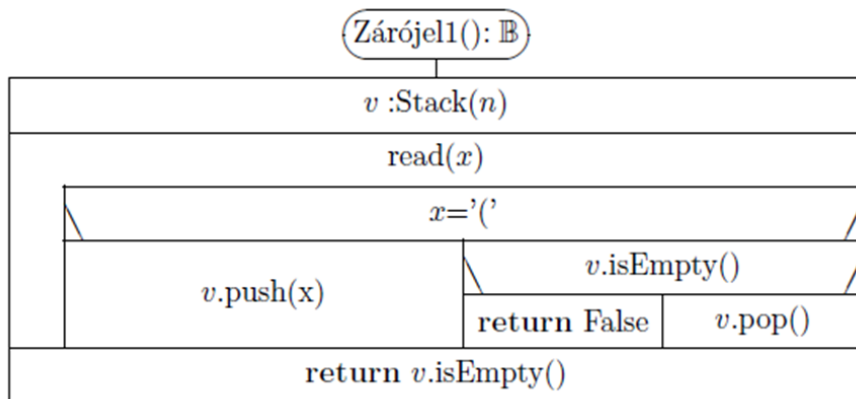
Some operations of a Stack



Veremmel megoldható problémák

Helyes zárójel-szintaxis

- Probléma: Adott egy sorozat zárójel, hogyan ellenőrizhetjük, hogy a szintaxisuk „helyes” vagy sem?
- Helyes alatt azt értjük, hogy minden *nyitó* zárójelhez tartozik egy később előforduló *záró* zárójel.
- Helyes: ((() ()))
- Nem helyes: (() VAGY ()))



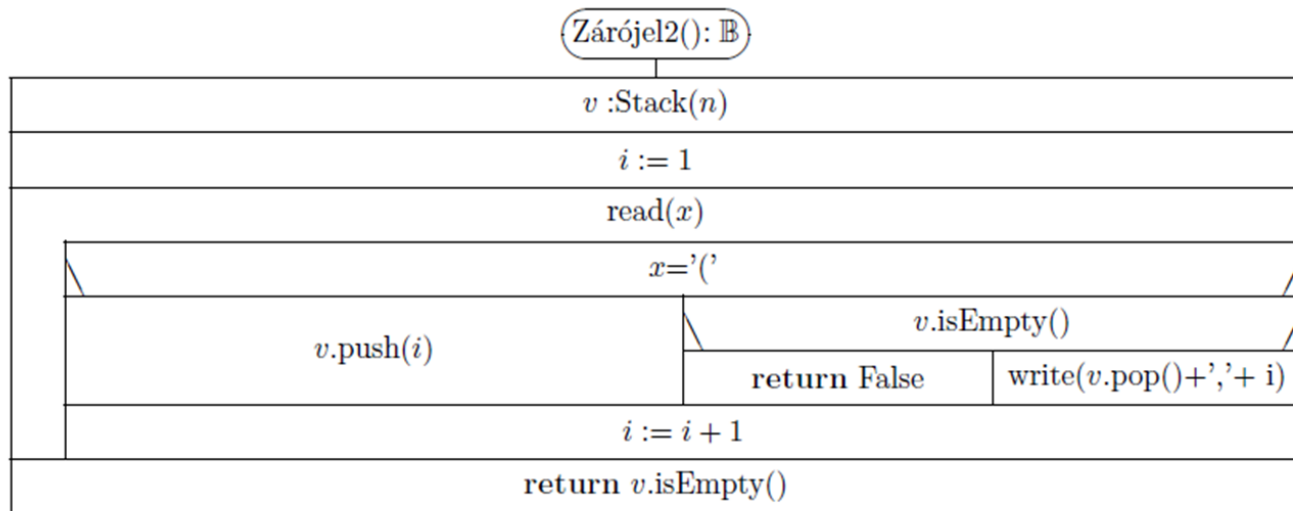
Számláló alapú megoldás

- Ehhez nincs szükség veremre, elegendő egy számláló.
- Kezdjük 0-ról.
- Nyitó zárójel esetén növeljük, záró zárójel esetén csökkentjük a számlálót.
- A kifejezés helyes, ha a számláló sosem lesz negatív, és a végére 0-t kapunk.

	(	(	)	(	)	)	(	(	(	)	(	)	)	)
0	1	2	1	2	1	0	1	2	3	2	3	2	1	0

Zárójelek jelölése

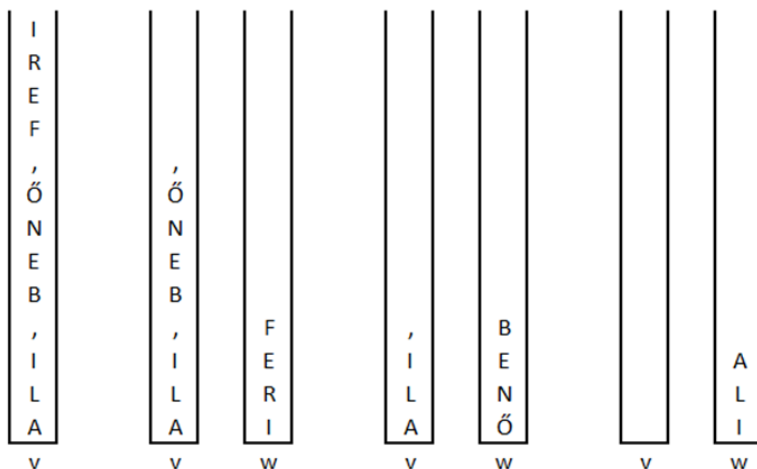
- Módosítsuk a veremalgoritmust úgy, hogy kiírja a zárójelpárok „azonosítóját”.



Nevek fordított sorrendben 1/4

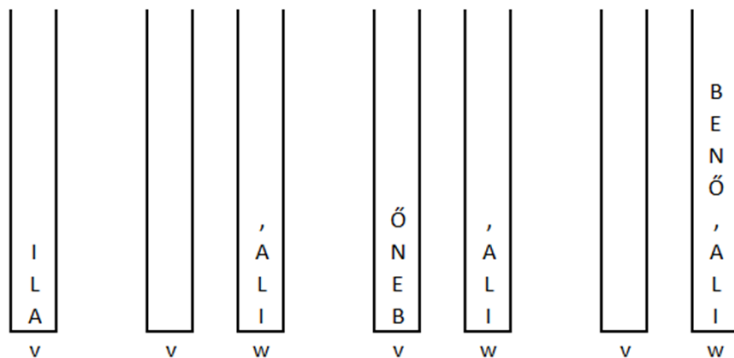
- Adott egy karakterenként olvasható, vesszővel elválasztott neveket tartalmazó sztring.
- A neveket fordított sorrendben szeretnénk kiírni.
- Példa:
 - Bemenet: Ali, Benő, Feri, Juci, Tomi
 - Kimenet: Tomi, Juci, Feri, Benő, Ali
- A megoldáshoz két veremet használhatunk.

Nevek fordított sorrendben 2/4



- Első megoldás:
 - Az egész bemenetet egy verembe olvassuk.
 - A verem tetejéről karakterenként áthelyezzük a második verembe a vesszőig, majd kiírjuk a második veremet.
 - Ismételjük, amíg az első verem ki nem ürül.

Nevek fordított sorrendben 3/4

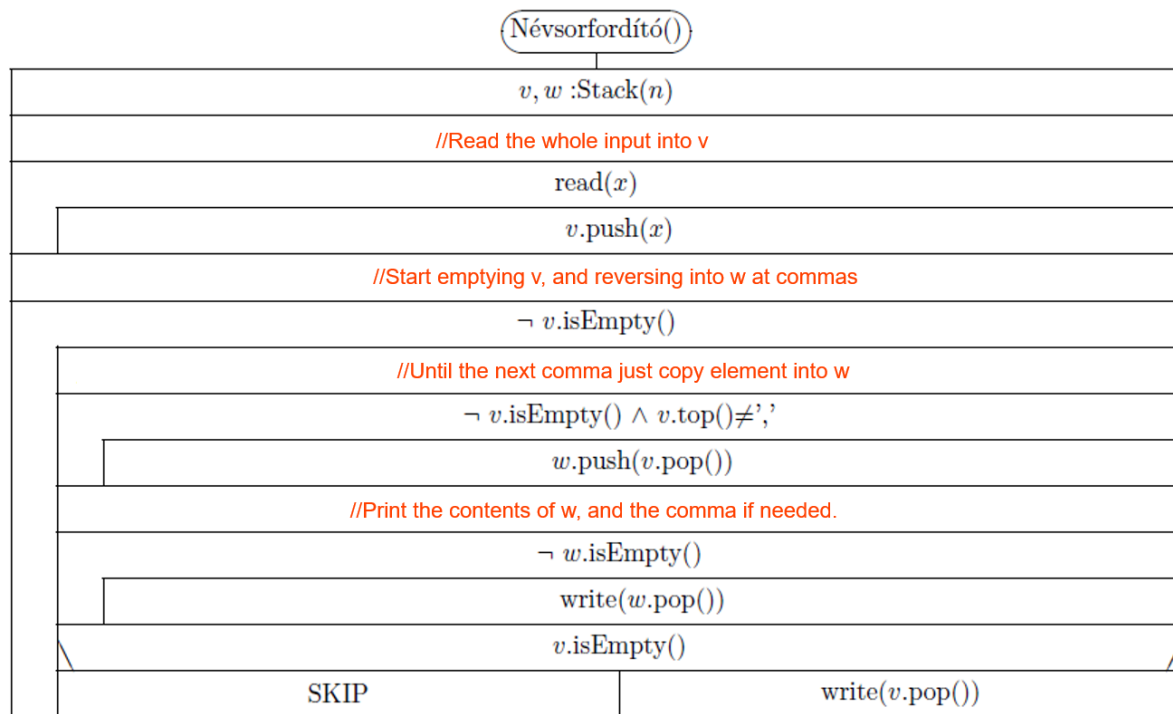


Kimenet: B E N Ő , A L I

- Második megoldás:
 - Az első verembe beolvassuk a bemenetet a vesszőig.
 - Az első vessző elérése után az első veremet átrakjuk a második verembe.
 - Ismételjük a bemenet végéig, majd kiírjuk a második veremet.

Nevek fordított sorrendben 4/4

- Az első megoldás struktogramja



- **Tükrözött-e a szöveg?** Adott egy legfeljebb n hosszú, betűkből és '#' szimbólumokból álló sorozat. A sorozatot tükrözöttnak nevezzük, ha felbontható olyan páratlan hosszú, palindrom karaktersorozatokról álló részekre, melyek középső karaktere egy '#' karakter. Egyszer végig olvasva a szöveget döntsük el, hogy tükrözött-e? Az algoritmus egyetlen vermet, és egyszerű segédváltozókat használhat.
- Példa:
 - Tükrözött: #, #####, abc#cba, ##a#aabc#cba
 - Nem tükrözött: abc, abc#cb, abc#cbaa#aa, ab#bac###c

Lengyel Forma

Műveletek sorrendje 1/2

- Mi a műveletek helyes sorrendje?
- A matematikai kifejezéseknek van egy adott precedenciája (sorrendje):
 - $a+b*c \rightarrow b*c \rightarrow a+(b*c)$
 - $a*b+c \rightarrow a*b \rightarrow (a*b)+c$
- Ezt a precedenciát zárójelekkel módosíthatjuk:
 - $a/(b-c)$
 - $(a*(b-c))/((x-y)*(x-z))$
- Bizonyos műveletek azonos precedenciaszinten állnak.

Műveletek sorrendje 2/2

- Néhány példa az infix $\rightarrow$ postfix átalakításra:
- $\langle in1 \rangle \langle op \rangle \langle in2 \rangle \rightarrow \langle in1 \rangle \langle in2 \rangle \langle op \rangle$

infix	postfix	Megjegyzés
$a+b$	$ab+$	műveleti jel az operandusai mögött áll
$a+b*c$	$abc*+$	műveletek rangsorának hatása: ($\text{prec}(*) > \text{prec}(+)$)
$a*b+c$	$ab*c+$	műveletek rangsorának hatása: ($\text{prec}(*) > \text{prec}(+)$)
$(a-b)/(c+d)$	$ab-cd+ /$	zárójelezés felülbírállhatja a műveletek rangsorát
$a*(b+c)$	$abc+*$	zárójelezés felülbírállhatja a műveletek rangsorát
$-a$	$a-$	Egyoperandusú operátorok (pl. negatív előjel)
$a/b*c$	$ab/c*$	azonos rangú műveletek általában balról jobbra sorrendben végzendők el
a^b^c	$abc^^$	a fenti szabály alól akad néhány kivétel, például az egymást követő hatványozás sorrendje jobbról balra értendő

Jan Łukasiewicz



- Eredetileg Jan Łukasiewicz lengyel matematikus javasolta az alternatív, prefix jelölést (1920).
- Később Charles Leonard Hamblin ausztrál számítógéptudós javasolta a *postfix* változatot (1950).
- Azonban a "Lengyel Forma" kifejezés megmaradt, így a postfix változatot is Lengyel Formának (vagy Fordított Lengyel Formának) hívjuk.

Fordított Lengyel Forma

- Egy matematikai jelölés algebrai kifejezések hatékony kiértékelésére (például egy fordítóprogram által).
- A megszokott „*emberi*” jelölésben, ha összeadjuk a 3-at és a 4-et, az **infix** jelölés szerint így írjuk:

$$3 + 4$$

ahol az **operátor** (+) a **operandusok** (3, 4) között áll.

- De miért ne írhatnánk úgy, hogy az operátor az operandusok után következzen? Ezt hívjuk **postfix** jelölésnek:

$$3\ 4\ +$$

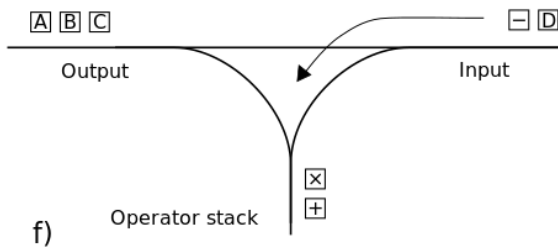
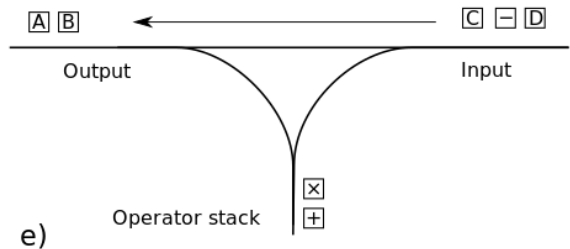
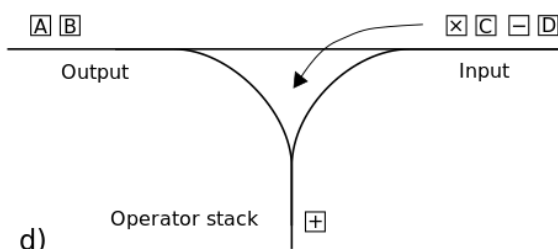
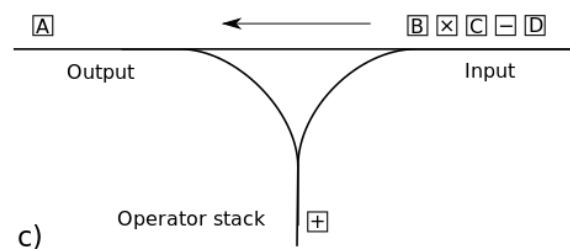
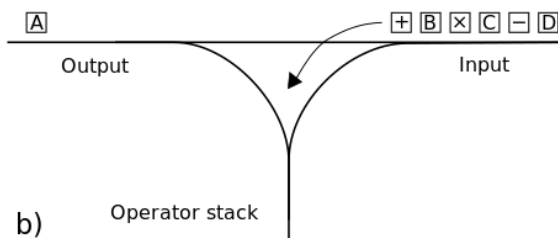
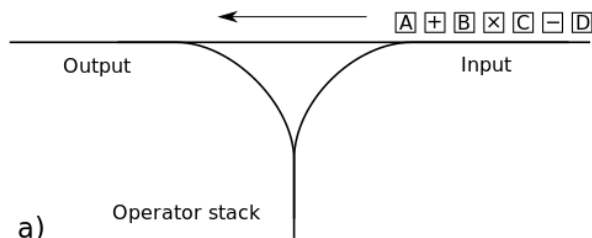
- Több művelet esetén például:

$$3 + 4 * 5 \rightarrow 3\ 4\ 5\ *\ +$$

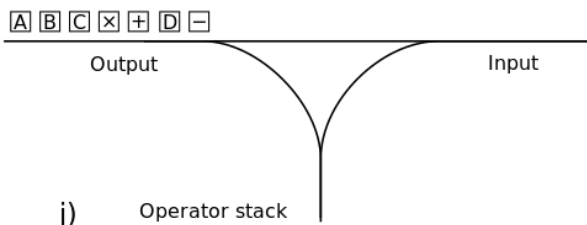
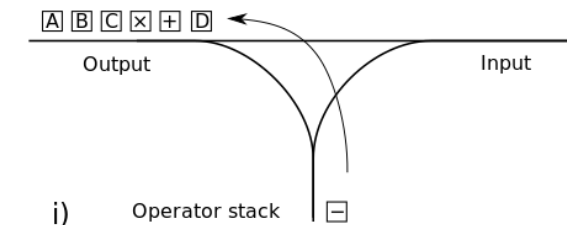
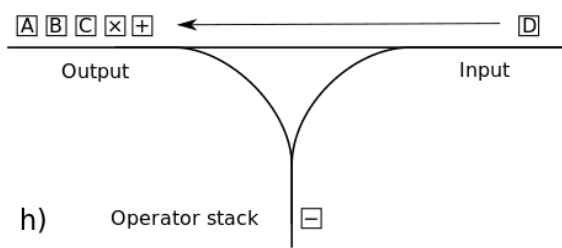
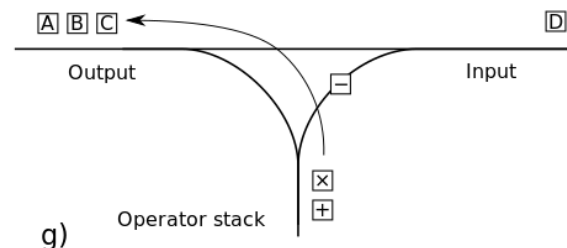
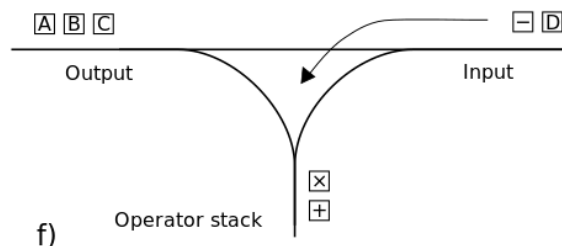
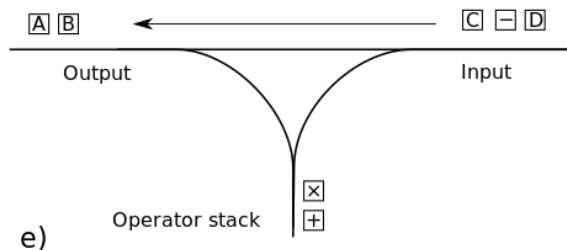
- Lengyel forma: egy aritmetikai kifejezés postfix alakja. Jellemzői:
 - Nincsenek benne zárójelek, a kiértékelés mégis egyértelmű, és könnyen elvégezhető.
 - Operandusok sorrendje nem változik, az infix kifejezéshez képest.
 - Operátorok sorrendje: az elvégzésük sorrendjében szerepelnek.
 - Minden operátort közvetlen megelőznek az operandusai, tehát amikor egy operátor megjelenik, biztosan az előtte lévő operandusokra vonatkozik.
 - Az operandus lehet változó, konstans, de lehet postfix kifejezés is.

- Dijkstra: Shunting Yard algoritmus
 - Egy módszer az aritmetikai vagy logikai kifejezések elemzésére, amely infix jelölést használ. Postfix kifejezést állít elő.
 - Az átalakításhoz két szöveges változót használunk: a *bemenetet* és a *kimenetet*.
 - Egy **verem** is szerepel, amely azokat az operátorokat tárolja, amelyeket még nem adtunk hozzá a kimeneti sorhoz.
 - Az átalakítás során a program sorban olvassa a karaktereket, és az aktuális szimbólumtól függően végrehajt egy műveletet.

Algoritmus grafikus ábrázolása



Algoritmus grafikus ábrázolása



Átalakítás infixből postfixbe

Original expression: $x = b + 2 / (a - c) - (-f \wedge b \wedge 2 * g) / z + w$

Precedence ranking:

$$\text{prec}(=) < \text{prec}(+) = \text{prec}(-) < \text{prec}(\ast) = \text{prec}(/) < \text{prec}(\wedge) \text{ prec}(-)$$

Contents of the stack: For each character whatever is currently in the stack

$$x = b + 2 / (a - c) - (- f \wedge b \wedge 2 * gg) / z + w$$

	=	+	/ +	(/	- /	/ -	(-	-	^ (	^ (	* (	- /	+ =	
--	---	---	-----	-----	-----	-----	-----	---	-----	-----	-----	-----	-----	--

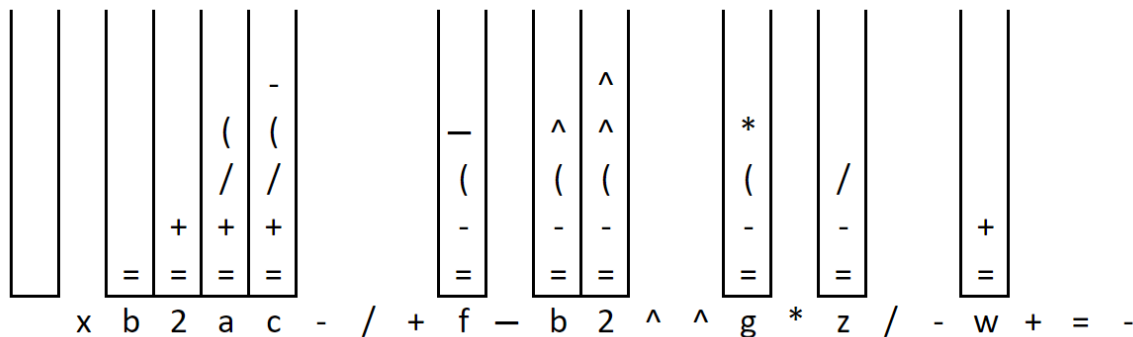
Resulting polish notation

$$x^2 + b^2 - a^2 - c^2 + f - b^2 \wedge \wedge g^* z / - w + =$$

Átalakítás infixből postfixbe

More compact if we just write the contents of the stack for every operand we write out.

Original expression: $x = b + 2 / (a - c) - (-f \wedge b \wedge 2 * g) / z + w$



Átalakítás infixből postfixbe

LengyelForma(S)				
V: Stack				
x := Read (S)				
x != ε				
Operandus(x)	x = ' ('	x = ') '	Operator(x)	
Write(x)	V.push (x)	V.top ≠ '('	BalJobbOperator(x)	
		Write(V.pop())	$!V.IsEmpty() \wedge V.top() \neq '('$ $\wedge pr(x) \leq pr(V.top())$	$!V.IsEmpty() \wedge V.top() \neq '('$ $\wedge pr(x) < pr(V.top())$
		V.pop()	Write(V.pop())	Write(V.pop())
		V.push(x)	V.push(x)	
x := Read (S)				
! V.IsEmpty()				
Write(V.pop())				

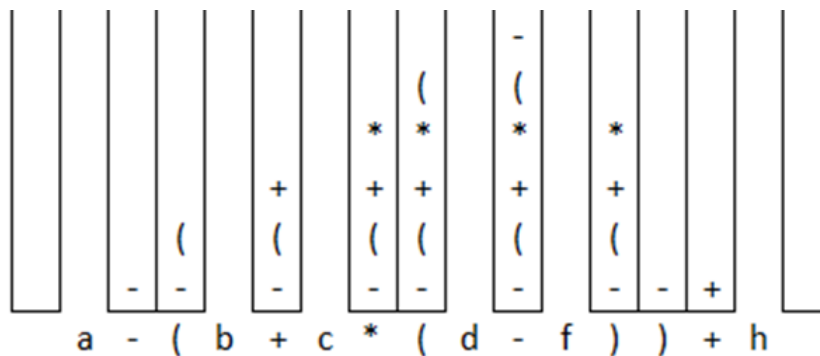
- LR operátorok: Balról jobbra értékelődő operátorok (+ - * / ...)
- RL operátorok: Jobbról balra értékelődő operátorok (^ := ...)

Átalakítás infixből postfixbe - Gyakorlás:

- Feladat:
 - $a - (b + c * (d - f)) + h$
 - kifejezés lengyel formára hozása verem segítségével. A verem tartalmát folyamatosan tartsuk nyilván!

Átalakítás infixből postfixbe - Gyakorlás:

- Feladat:
 - $a - (b + c * (d - f)) + h$
 - kifejezés lengyel formára hozása verem segítségével. A verem tartalmát folyamatosan tartsuk nyilván!
- Megoldás:



Fordított Lengyel Forma kiértékelése

- Legyen adott a következő kifejezés: $(a + b) / c - d^2$
- Ennek a Fordított Lengyel Formája: $a\ b\ +\ c\ /\ d\ ^2\ -$
- Legyenek a változók: $a=12$, $b=8$, $c=2$, $d=3$. Számoljuk ki az értékét!
- Módszer:
 - Sorban beolvassuk a következő karaktert.
 - Ha operandus, betesszük a verembe.
 - Ha operátor, kivesszük a megfelelő operandusokat, végrehajtjuk a műveletet, majd visszatesszük az eredményt.
 - A végén a veremben maradt egyetlen elem lesz a végeredmény.

Fordított Lengyel Forma kiértékelése

$a \ b \ + \ c \ / \ d \ 2 \ ^ \ -$

$a=12$

$b=8$

$c=2$

$d=3$

						2		
	8		2		3	3	9	
12	12	20	20	10	10	10	10	1
a	b	+	c	/	d	2	^	-

Fordított Lengyel Forma kiértékelése

lengyel_kiertekeles(n:N)

V:Stack	
read(x)	
Operator(x)	Operandus(x) V.push(x)
jobb:=V.pop()	
bal:=V.pop()	
V.push(bal ⊗ jobb)	
write(V.pop())	

Bemenet - lengyel formájú kifejezés

Megjegyzések az Operator(x) ághoz:

- bal $\otimes$ jobb jelölés: elvégezzük az x műveletet,
- feltettük, hogy az operátorok két operandusúak, de ez könnyen kiterjeszthető egy operandusú műveletekre
- az algoritmus kiszámíthatja az értéket, vagy fordító program esetén generálhatja a kiszámítás kódját.

Lengyel forma gyakorlás:

- Feladat:
 - Hozzuk lengyel formára a következő kifejezést, majd verem segítségével értékeljük ki:
 - $(a + b) * c - d$
 - Lengyel forma: $a b + c * d -$
 - Tegyük fel, hogy a változók az alábbi értékkel rendelkeznek, számítsuk ki a kifejezés értékét a lengyel formából!
 - $a = 2, b = 4, c = 3, d = 1$

Lengyel forma kiértékelése - Gyakorlás:

- Feladat:
 - Hozzuk lengyel formára a következő kifejezést, majd verem segítségével értékeljük ki:
 - $(a + b) * c - d$
 - Lengyel forma: $a \ b \ + \ c \ * \ d \ -$
 - Tegyük fel, hogy a változók az alábbi értékkel rendelkeznek, számítsuk ki a kifejezés értékét a lengyel formából!
 - $a = 2, b = 4, c = 3, d = 1$
- Kiértékelés:

	4		3		1	
2	2	6	6	18	18	17
2	4	+	3	*	1	-

Köszönöm a figyelmet!